



Every time we travel, we see some traffic signals. Have you ever wondered how they work?



Every day, we travel using different vehicles. Some days we use cars or motorcycles, other days we use bicycles or auto rickshaws. A commonality between all these vehicles is that they have to follow traffic signals. Even pedestrians have to follow them.

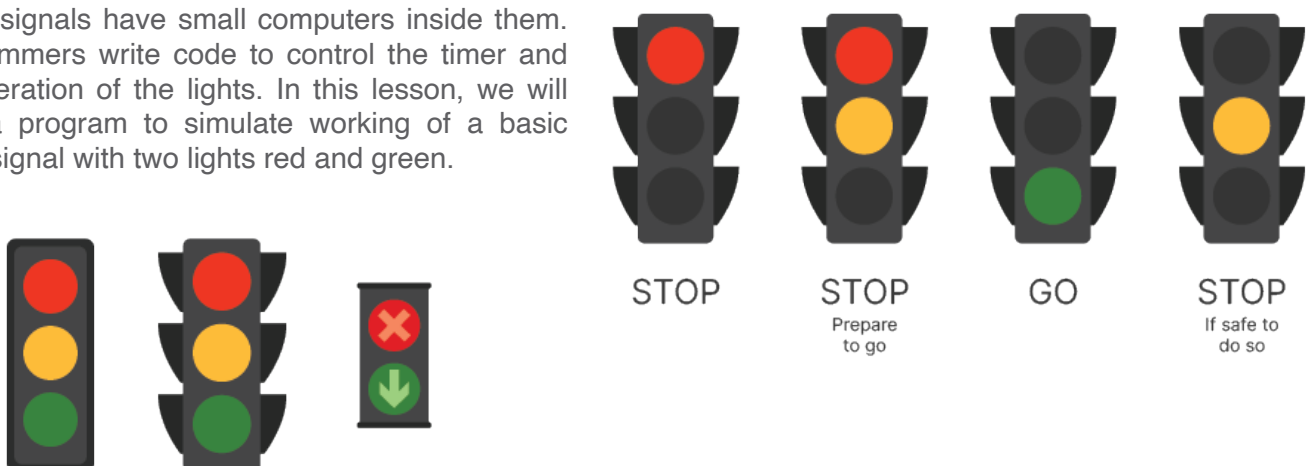


Traffic signals play an important part in our transportation system. They control traffic and help us avoid accidents. They use different coloured lights which tell vehicles and people to STOP and GO. Traffic signals are not only used on streets, but also on train tracks.



A red coloured signal indicates that the vehicle should stop, and a green signal tells the vehicle to go ahead. Many traffic signals, especially in heavy traffic areas, have an orange signal which tells the vehicle to reduce its speed and prepare to stop. Traffic signals also have a counter which displays the seconds remaining for a signal colour to change.

Traffic signals have small computers inside them. Programmers write code to control the timer and the operation of the lights. In this lesson, we will write a program to simulate working of a basic traffic signal with two lights red and green.



Activity

Write a code in p5.js to operate a traffic signal with a timer.

We will follow the given sequence of steps:

- Use code to draw the signal using various shapes.
- Learn about timers and add functionality of the timers to update the signal.
- Code to update the traffic lights.

1 Draw a background for the traffic signal

- Open p5js editor to start a new activity. Add **noStroke** command to draw shapes without a border.
- Add 'rectMode(CENTER);' command to draw centre aligned rectangles.
- Draw a background sky using a rectangle at "x=width/2" and "y=height/2" and dimensions to fill the canvas.



```
1 function setup() {  
2   createCanvas(400, 400);  
3 }  
4  
5 function draw() {  
6   background(220);  
7  
8   //Drawing Background  
9   noStroke();           //no border  
10  rectMode(CENTER);  
11  
12  fill(180,230,255);     //Sky blue color  
13  rect(width/2,height/2,width, height);  
14  
15 }
```

Activity

d. Draw land in the background using another rectangle, such that it covers the lower half of the canvas.

```

> sketch.js
2 createCanvas(400, 400);
3 }
4
5 function draw() {
6   background(220);
7
8   //Drawing Background
9   noStroke();           //no border
10  rectMode(CENTER);
11
12  fill(180,230,255);     //Sky blue color
13  rect(width/2,height/2,width, height);
14
15  fill(200,200,100);     //Land color
16  rect(width/2,3*height/4,width,height/2);
17
18 }
19
20

```

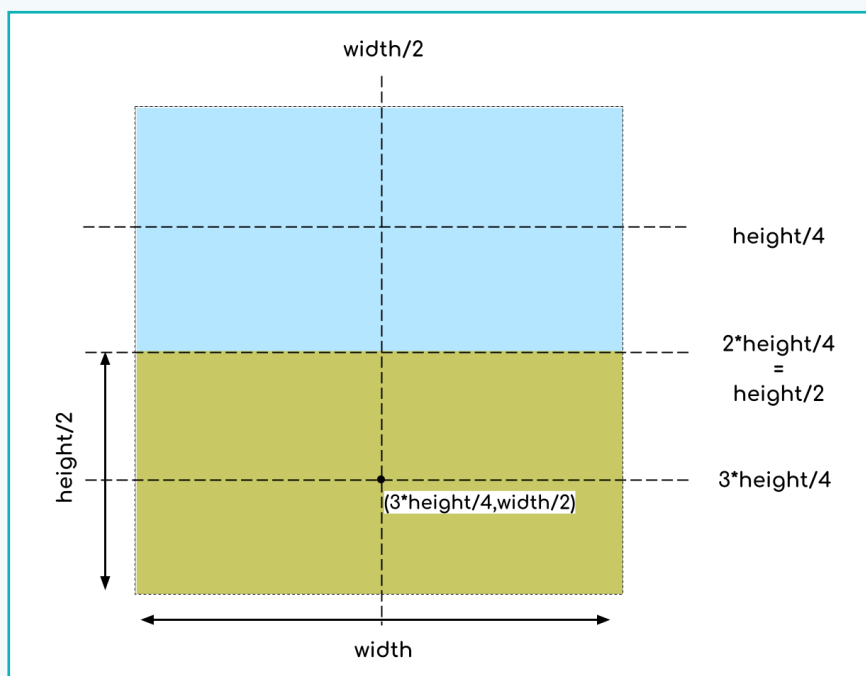
Preview

Console

Clear

Note:

With `rectMode(CENTER);` command, the x and y values in `rect()` command are at the center of the rectangle. Also note the following diagram showing dimensions for further positioning of the signal elements.



Activity

2 Draw a structure for a signal

a. Draw a vertical rectangle as a pole for the signal.



b. Draw a square at the centre of the canvas for the body.





Recall:

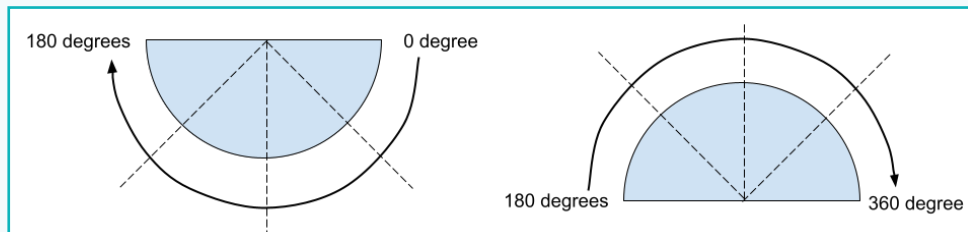
`arc(x, y, width, height, start_angle, end_angle)` command is an extension of the ellipse command, where we provide a start angle and an end angle to draw an arc. With this command, a part of the ellipse is shown similar to a pie or pizza piece.

Note:

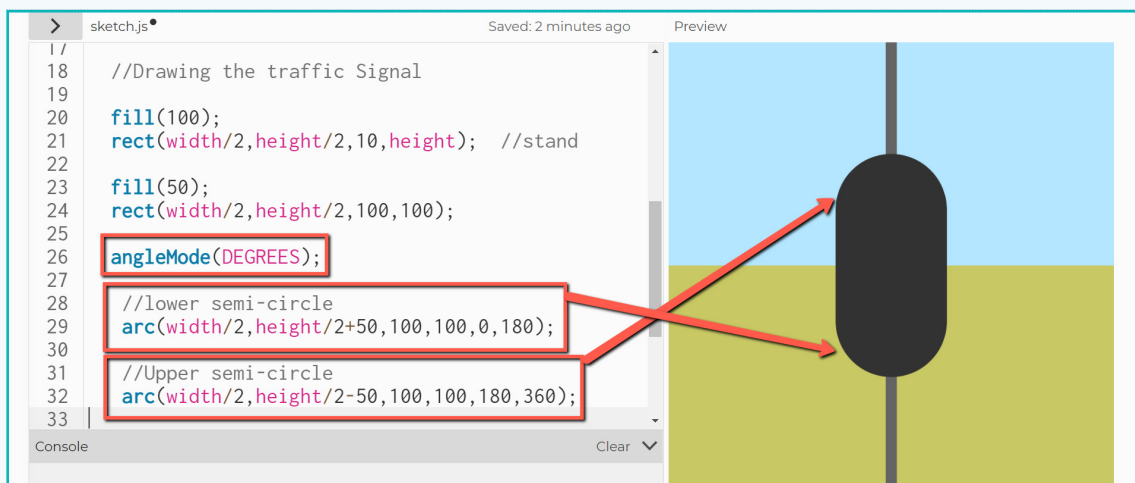
Angle measurement in p5.js is in a clockwise direction, unlike regular geometry where it is anticlockwise.

Activity

- c. To draw a lower and upper semi-circle, we will draw 2 semi circles from 0 to 180 and 180 to 360 degrees.

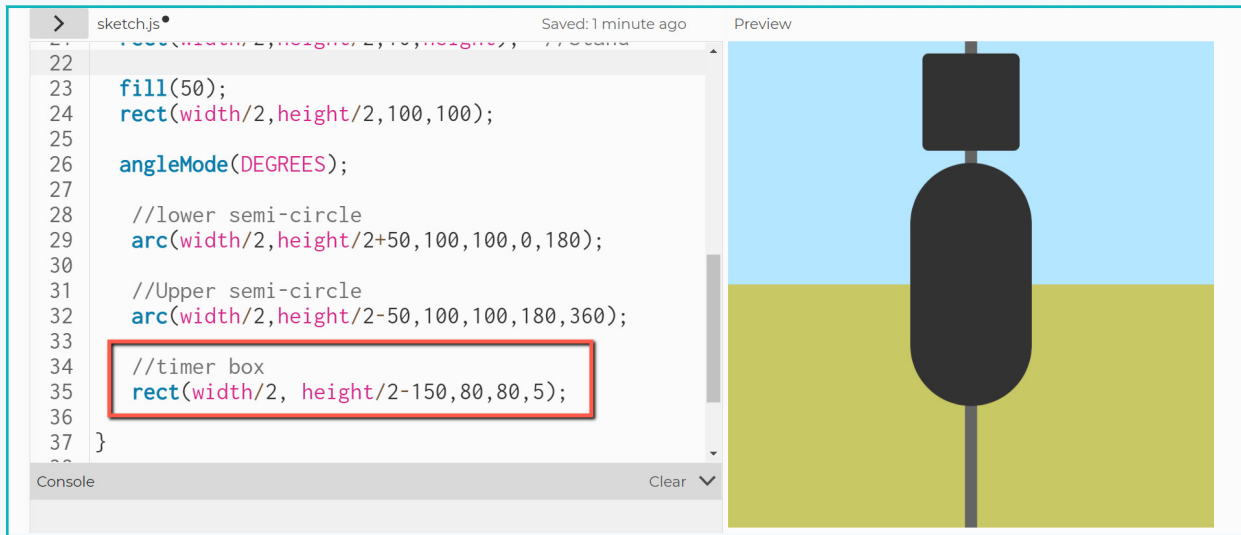


- d. Use `arc()` command to draw the upper and lower curved part of a signal using two semicircle shapes.



Activity

e. Draw a square with rounded borders above the signal. This will display the timer value.



Later, we will add a time counter in the square.

3 Draw traffic lights

- Draw two circles as red and green lights.
- Use fill() command to colour one light red and other green.



Activity

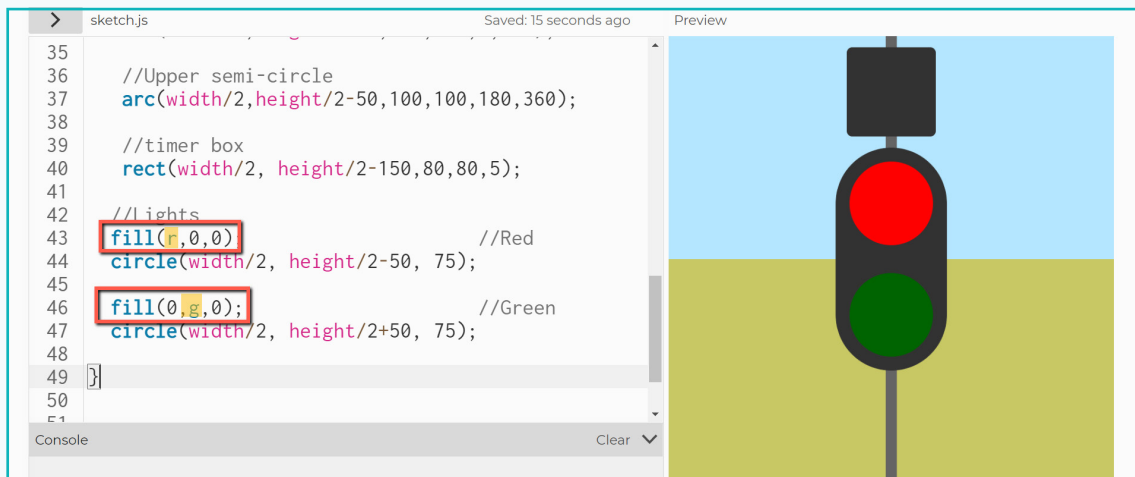
4 Add a variable for signal colours

We will change the colours of the light circles to show them as switched on or off. To display a bright red or green light, the colour value should be 255. To show a switched off light, the colour value will be 100. Since the colour values are changing, we will need to use variables to code.

- Declare two variables 'r' and 'g' at the top of the code.
- Assign initial values to the variables 'r' and 'g' in the setup function as 255 and 100 respectively.



- Add variables in the fill() commands for their respective coloured circles. We will use variables here instead of values.



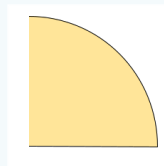
Activity

In a signal at any given time, only one of the red and green lights should be switched on. The other one should be off. If we exchange the values of 'r' and 'g' variables, then the light that was on will turn off, and other one will be switched on. We can repeat the process of exchanging values inside a variable and alternate which light switches on.

Exercise

Tick mark ☒ the correct answer

- 1** To draw the following arc, angle1 and angle2 values in command `arc(x,y,length,height,angle1,angle2)` should be _____. (Code and then answer)



- | | | | |
|----------------------|--------------------------|-----------------------|--------------------------|
| a. 0 to 90 degrees | ☐ | b. 270 to 0 degrees | ☐ |
| c. 90 to 180 degrees | ☐ | d. 180 to 270 degrees | ☐ |

- 2** _____ parameter in the `rect` command decides the radius of a rounded square's corner.

- | | | | |
|--------|--------------------------|--------|--------------------------|
| a. 1st | ☐ | b. 3rd | ☐ |
| c. 4th | ☐ | d. 5th | ☐ |

- 3** To draw a shape at the centre of the canvas, what should be the x and y location using default variables?

- | | | | |
|---|--------------------------|---|--------------------------|
| a. $x = \text{width}$, $y = \text{height}$ | ☐ | b. $x = \text{width}$, $y = \text{height}/2$ | ☐ |
| c. $x = \text{width}/2$, $y = \text{height}/2$ | ☐ | d. None of above | ☐ |

The concept of exchanging values inside variables is often used in programming. Before programming, the logical sequence of such tasks is decided. A logical sequence like this is called an algorithm. Let's learn about algorithms in detail.

Algorithm

An algorithm is a step-by-step procedure of how to perform a specific task to solve a specific logical or mathematical problem.

For example: we have a set of numbers

[3, 4, 6, 2, 5, 7]

We can create a general set of logical commands to arrange these numbers in an ascending order.

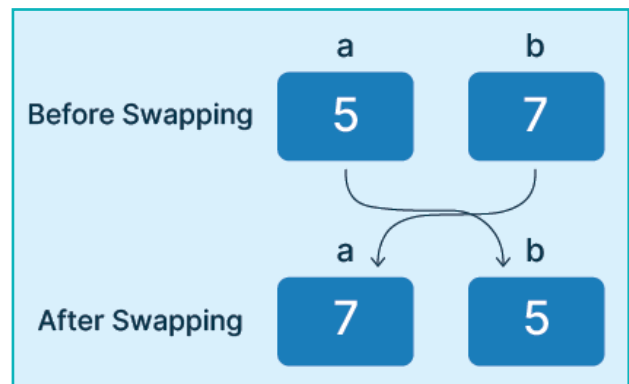
[2, 3, 4, 5, 6, 7]

This set of generic steps to achieve this for any sequence will be an algorithm, which is used to write code. The same algorithm can be used to arrange any other set of numbers in an ascending order.

We will learn about an algorithm which can swap values between variables.

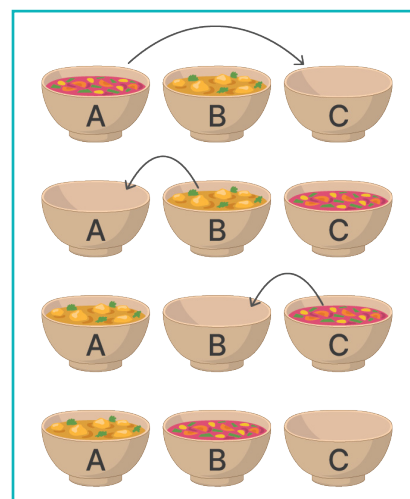
Swap Algorithm

We can use the swap algorithm to exchange or swap values stored in two variables.



It typically involves three steps in coding:

1. Store the value of the first variable in a temporary variable.
2. Assign the value of the second variable to the first variable.
3. Assign the value stored in the temporary variable to the second variable.



This is the simplest way. There are other methods as well but we will focus on this one for now.

Activity

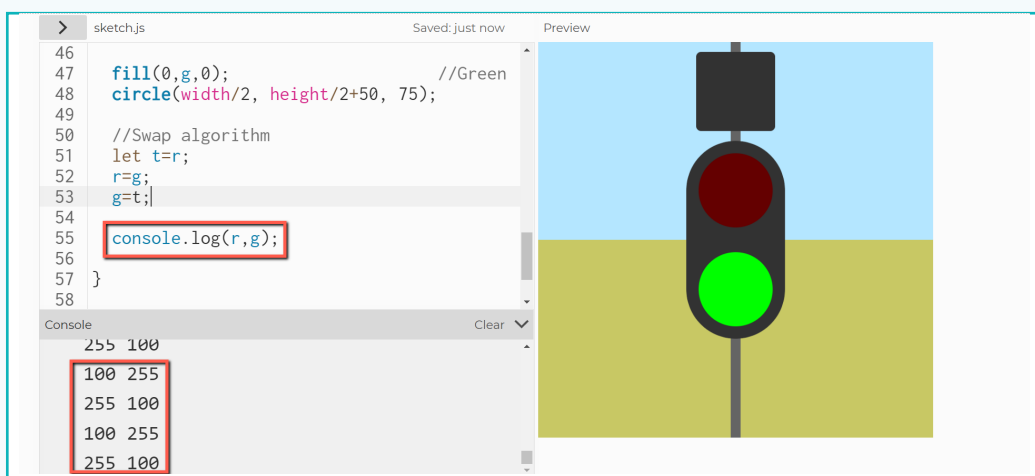
5 Use the swap algorithm to change traffic signal colours

- Create a temporary variable 't' and assign the value of 'r' to 't'.
- Assign the value of 'g' to 'r'.
- Assign the value stored in temporary variable 't' to 'g'.



Now the lights will switch on and off continuously as per the draw() function. The switching happens too fast and is not visible. But we can use the console to check if it is working correctly.

- Add console.log(r,g) command to check if the swap operation is taking place.



Even though variable values are swapped, right now we see both lights switched ON. To avoid this, let's add a delay between the transition using millis() command.

Exercise

State whether True or False

4 Algorithm is the step-by-step code syntax we write for a computer to perform a specific task.

5 The swap algorithm is used to exchange values inside variables.

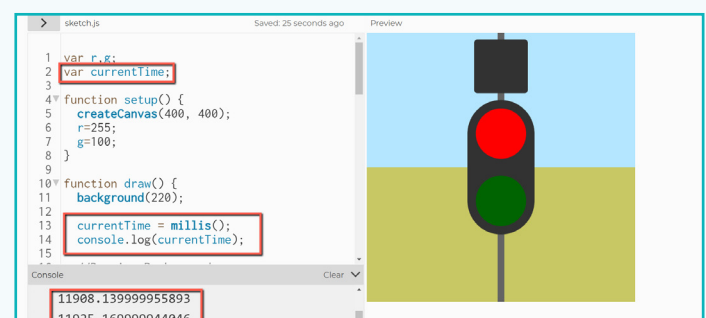
6 A temporary variable is used in the swap algorithm to temporarily store the value of one of the variables.

millis() command returns the number of milliseconds (1000 milliseconds = 1 second) since the code starts running i.e. when **setup()** is called. This is used to time events and animation sequences.

Activity

Create a variable “currentTime” and assign the **millis()** function to it for timing the signals.

- Declare a variable called **currentTime** at the beginning of the code.
- Assign the value of time elapsed to the variable. Use **millis()** command in the **draw()** function since it needs to be continuously updated.
- Comment out other **console.log()** commands and add a new one to check the value in the **currentTime** variable.

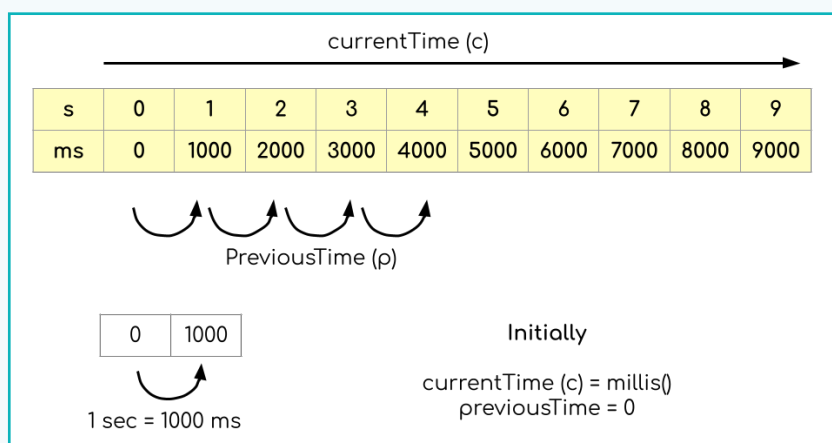


Activity

As the code starts executing, variable “currentTime” displays the updated value of time lapsed.

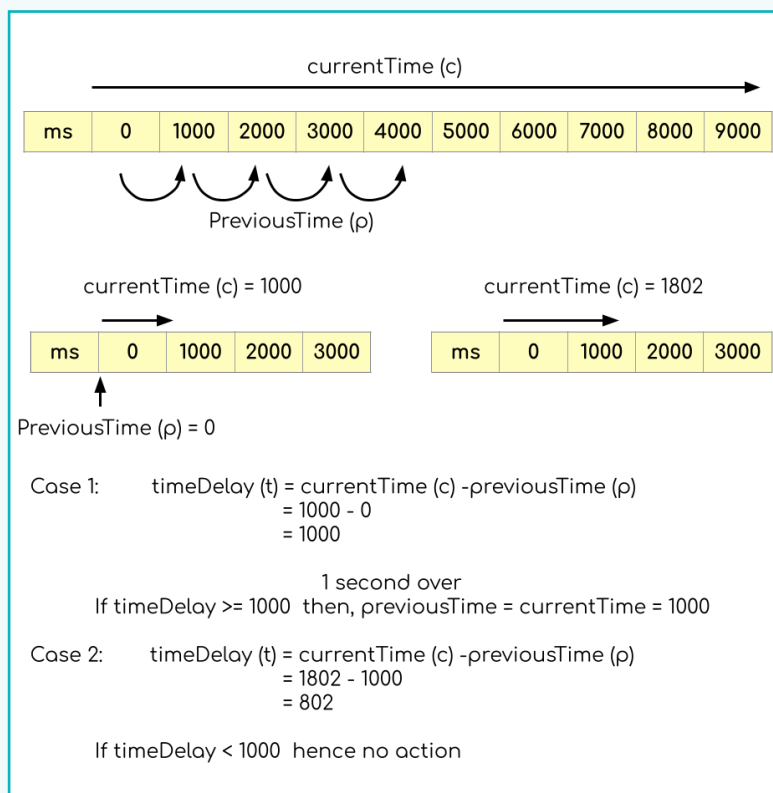
We can check if time elapsed is greater than 1 second, i.e. 1000 milliseconds. After the first second, we will keep track of every second.

Create another variable called previousTime with an initial value of 0 and check the difference between the two variables after each second elapsed.



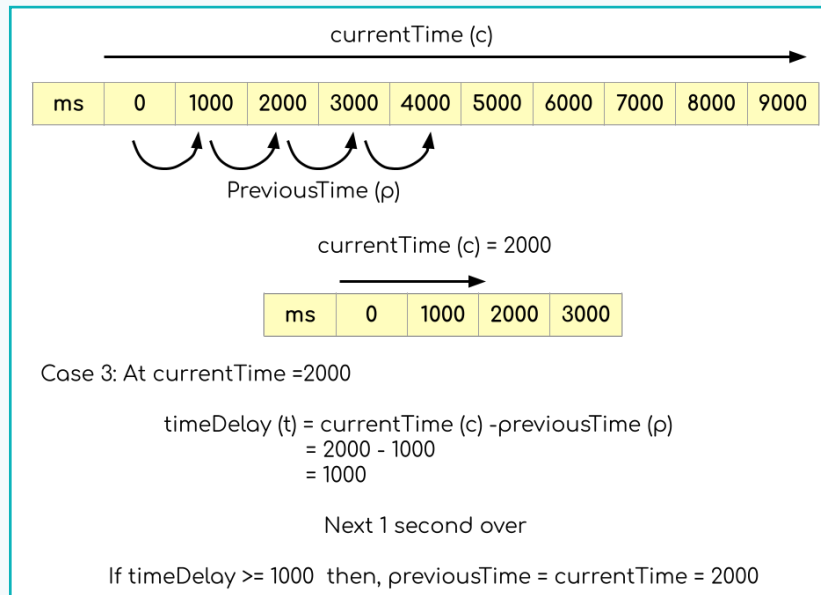
currentTime value continuously increases as the code runs. The initial value in the previousTime variable is set to 0. Save the difference between two variables (currentTime – previousTime) in a new variable called timeDelay.

If timeDelay $\geq$ 1000 (1000 ms), then 1 second is over. To check when the next second has passed, update the value in previousTime to 1000.

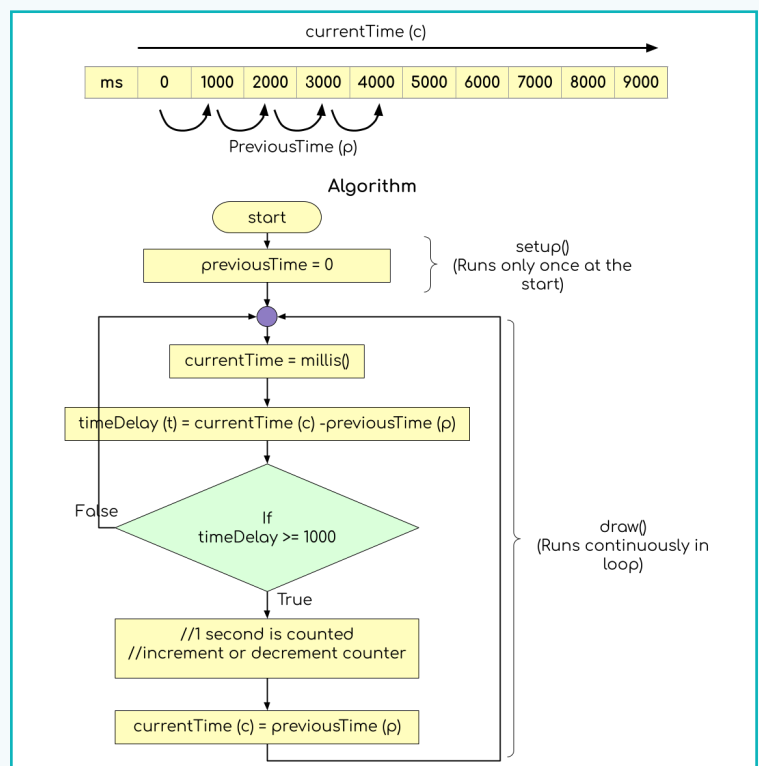


Activity

In this way we can count each second that passes by updating the previousTime variable when 1000 ms have passed. Every time a second passes, we can update the counter to keep track of time.



We have already learnt about flowcharts that create logic and algorithms for code. We can create a flowchart for this logic.



Activity

6 Add a timer using the millis() function

- Declare variables 'previousTime' and 'timeDelay'.
- Set their initial value to 0 in setup() function.

```
sketch.js Saved: 25 seconds ago
1 var r,g;
2 var currentTime;
3 var previousTime;
4 var timeDealy;
5
6 function setup() {
7   createCanvas(400, 400);
8   r=255;
9   g=100;
10
11   previousTime=0;
12   timeDealy=0;
13 }
14
15 function draw() {
16   background(220);
17 }
```

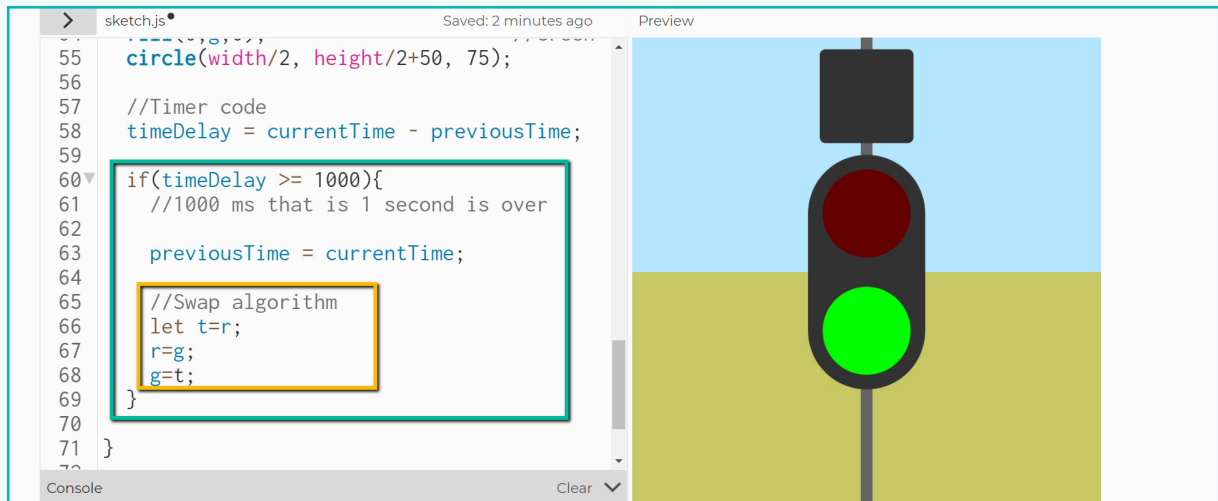
- Assign value for timeDelay as (currentTime – previousTime) before the swap algorithm code.
- Use if condition to check if timeDelay value is greater than or equal to ($\geq$) 1000 ms i.e.1 second.
- If the condition is satisfied, set value of previousTime variable to currentVariable

```
sketch.js Saved: 1 minute ago
53
54 fill(0,g,0); //Green
55 circle(width/2, height/2+50, 75);
56
57 //Timer code
58 timeDelay = currentTime - previousTime;
59
60 if(timeDelay >= 1000){
61   //1000 ms that is 1 second is over
62
63   previousTime = currentTime;
64 }
65
66 //Swap algorithm
67 let t=r;
68 r=g;
69 g=t;
70 }
```

Now our code detects every single second that passes. We can use this to change the signal colours each second.

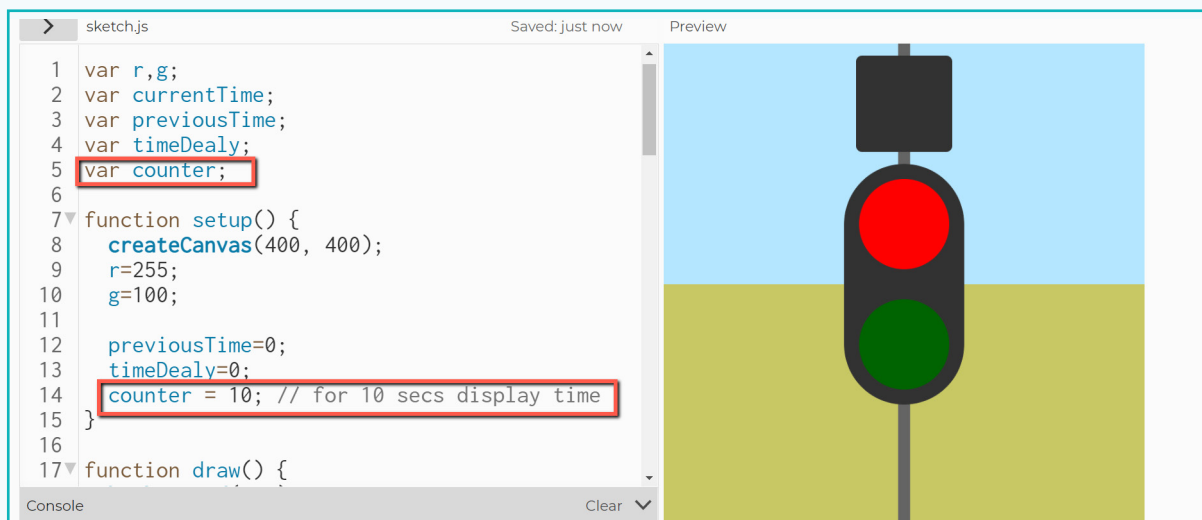
Activity

- f. Move the swap algorithm code inside 'if' condition to check the value in the timeDelay variable.



The red and green lights switch on alternately after every second. But usually, one light at a signal is on for more than 10 or 20 seconds. We can use a counter variable to count the number of seconds and increase the signal time.

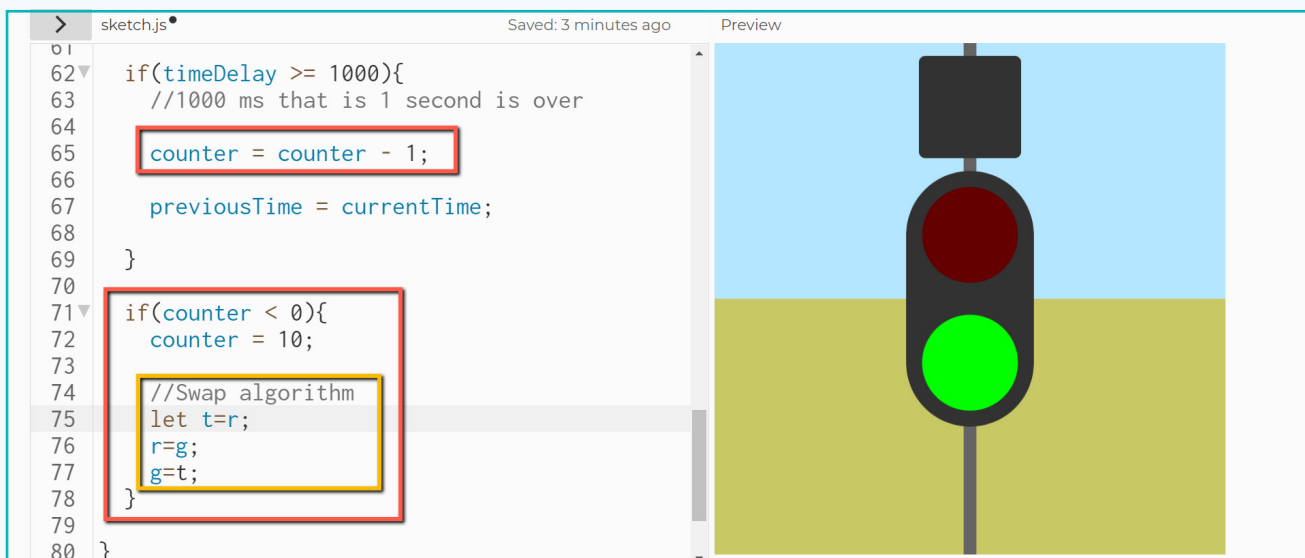
- g. To update the code, undo the previous step of moving swap algorithm in if condition.
- h. Declare a counter variable at the top of the code and set the initial value to the required time. For example, we can set counter variable value to 10 for signal display time of 10 seconds.



Activity

With every second that passes, we can decrease the value of the counter variable by 1. If value becomes 0, we can reset it to 10 using if condition. This way, we can change the signal lights after 10 seconds instead of just 1.

- i. Decrease the value of counter variable by 1 after every second by adding it in if condition.
- j. Create another if condition to check if counter value is less than 0. If yes, set it back to 10.
- k. Check if the counter value has updated after every 10 seconds and move code for the swap algorithm in if condition.



```
61
62 if(timeDelay >= 1000){
63   //1000 ms that is 1 second is over
64   counter = counter - 1;
65   previousTime = currentTime;
66 }
67
68
69
70
71 if(counter < 0){
72   counter = 10;
73
74   //Swap algorithm
75   let t=r;
76   r=g;
77   g=t;
78 }
79
80 }
```

Activity

7 Adding Counter Display

Now we can see that the signal changes colour after every 10 seconds. We can also display the counter in the square box at the top, just like in real traffic signals.

- Add fill() command to set text colour.
- Add textSize(); command to set a text size suitable to square box.
- Add textAlign(CENTER); command to centre align text to the display.
- Use text(text value, x position, y position); command to display counter digits. Text value will be the value of the counter variable, and x and y position are used to keep the text in the centre of the square.



With this, our traffic signal simulation is complete. Run the code to check if it is working correctly. Try using different timer values for signals and separate timers for red and green lights.

Exercise

Answer in one line

7 Which command displays time elapsed in milliseconds after the code executes?



8 To check the passing of 1 second, what should the difference between `currentTime` and `previousTime` variables be greater than or equal to?



9 Which command and parameters are used to display text?



A similar timer logic is implemented in various devices that we use in our day-to-day life.



Tech Flash

- **Algorithm** : A step-by-step procedure of how to perform a specific task to solve a specific logical or mathematical problem.
- **Swap Algorithm** : An algorithm that swaps values in two variables.
- **Timer count down** : Sets a specific time and sends an alert when that time is up.